

even the most advanced programmers commit memory related errors. This makes Valgrind a tool that's sought after even by the most seasoned programmer. The working of Valgrind is explained with the help of the program *buggy_program2.c* given below.

```
/* Program name : buggy_program2.c */
#include<stdio.h>
#include<stdlib.h>
int main( )
{
    char *ptr=calloc(5,sizeof(char));
    ptr[5]='A';
}
```

This program is compiled to get the executable by typing the command *gcc program_name.c*. The executable created is used by the Valgrind utility to identify memory related vulnerabilities. There are multiple tools in Valgrind. The default and most frequently used one is Memcheck. The command for invoking Valgrind is *valgrind executable_name*. Valgrind will identify both the memory related errors in the program, the invalid write at location *ptr[5]* and the memory leakage of 5 bytes caused by not calling *free()*.

Profiling tools

Profiling tools are used to analyse the performance of a program. They are used to analyse both the space and time complexity of a program. They also carry out a form of dynamic code analysis. The most widely used performance analysis tool for C is Gprof, an extended version of an older tool called *prof*. Profiling should be enabled while compiling the programs as follows. A program called *gprof_example.c* is used to analyse how Gprof works.

```
/* Program name : gprof_example.c */
#include<stdio.h>
void delay1( )
{
    int j;
    for(j=0;j<10000000;j++);
    printf("\nIn function delay1\n");
}

void delay2( )
{
    int k;
    for(k=0;k<10000000;k++);
    printf("\nIn function delay2\n");
}

int main( )
{
    int i;
    for(i=0;i<10000000;i++);
    printf("\nIn function main\n");
    delay1( );
    delay2( );
}
```

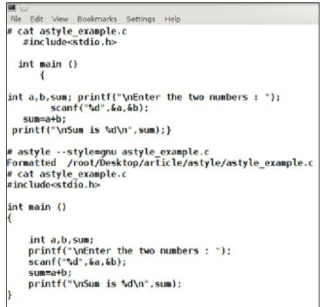


Figure 2: Beautifying programs with Astyle

```
return 0;
}
```

The Gprof utility should be enabled while compiling the program as follows: *gcc -p program_name.c*. Before calling Gprof, the program should be executed once. This will create a file called *gmon.out*. This, along with the executable file, is used by the Gprof utility to analyse the performance of the program. The command to call Gprof is: *gprof a.out gmon.out*. The time taken by various functions can be easily understood from the output of Gprof. If the program *gprof_example.c* is processed with Gprof, then you will observe that the execution time for function *delay2()* is roughly 10 times that of function *delay1()*. This sort of information can be further used to identify functions that require optimisation.

Code formatters

There are programming languages like Python and Occam in which the formatting style of the program is as important as the syntax to get the correct output. Such languages force the programmer to write well intended programs. But C is a free-form language, which does require proper formatting to obtain the correct output. The result of such freedom, if misused, often gives nightmares and sleepless nights to many a programmer. It is very difficult to debug programs if the code is not properly intended. Many different styles have been proposed for formatting C programs. Some of the very popular ones are K&R style, GNU style, Kernel style, etc. Problems arise when programmers with different formatting styles work together on a project. This problem can easily be solved by using a code formatter, which can translate code from one style to another. The most widely used code formatter in the Linux environment is Astyle. It supports multiple styles like Allmann, KR, GNU, etc, which are suitable